__/__/__

DSA

→ Asymptotic Notation:-

1. Big O notation:- (Upper Bound) $f(n) \leq c \cdot g(n) \; \forall \; n \geq n_0$
2. Big Ω notation:- (lower Bound) $f(n) \geq c \cdot g(n) \; \forall \; n \geq n_0$
3. Theta notation:- (Tight Bound) $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \; n \geq n$
4. small o notation:- $f(n) < c \cdot g(n) \; \forall \; n \geq n_0$
5. small ω notation:- $f(n) > c \cdot g(n) \; \forall \; n \geq n_0$

Properties:-

1. Reflexive:- O, Ω, Θ
2. Symmetric:- Θ
3. Transitive:- $O, \Omega, \Theta, o, \omega$
4. $f(n) = O(g(n))$ iff $g(n) = \Omega(f(n))$
5. $f(n) = O(g(n))$
   $d(n) = O(e(n))$
   $f(n) + d(n) = O(\max(g(n), e(n)))$
6. $f(n) = O(g(n))$
   $d(n) = O(e(n))$
   $f(n) \times d(n) = O(g(n) \times e(n))$

→ $e^n > 2^n > n^2 > n\sqrt{n} > n\log n > n > 1/n$

$\log(n!) \approx \Theta(n\log(n))$

→ Space complexity = Total space = I/P + $\underbrace{\text{Extra space}}_{S(n)}$

$S(n) = O(T(n))$

→ Master Method:- $T(n) = aT(n/b) + f(n)$ $\quad a \geq 1, b \geq 1$

$f(n) = \Theta(n^k \log^p n)$

Case 1:- if $\log_b a > k \Rightarrow \Theta(n^{\log_b a})$

Case 2:- if $\log_b a = k$

if $p > -1 \rightarrow \Theta(n^k \log^{p+1} n)$
$p = -1 \rightarrow \Theta(n^k \log\log n)$
$p < -1 \rightarrow \Theta(n^k)$

Case 3:- if $\log_b a < k$

if $p \geq 0$, $\Theta(n^k \log^p n)$
$p < 0$, $\Theta(n^k)$

→ Binary Tree using array:-

'n' nodes B.T. ⇒ array size n to $2^n - 1$
parent index = i
then child index = $2i$, $2i+1$ [if index starts with 1]

→ Heap:- 1. almost complete n-ary tree.
2. must satisfy heap property.
Types:- Min heap, Max heap (default)
array implementation best for heap.

Leaf nodes = $\left\lceil \frac{n}{2} \right\rceil$ [$n \rightarrow$ nodes in heap]

Internal nodes = $\left\lfloor \frac{n}{2} \right\rfloor$

→ A[lb ... ub] size of each element is c byte
& x is BA. LOC[A[i]] ⇒ BA + $(i - lb) \times c$

for lower Δ matrix:- (Row Major)

$$loc(A[i][j]) = c \times \left(\frac{(i-1-lb+1)(i-1-lb+1+1) + (j-1-lb+1)}{2} \right) + BA$$

→ No. of BST with n nodes & n levels that contain no. from 1 to n w/o repetition = 2^{n-1}

	Min element			Suc. Search			Unsuc. Search			Levels		
	BC	WC	AC	BC	WC	AC	BC	WC	AC	BC	WC	AC
BST	$\Theta(1)$	$\Theta(n)$	$\Theta(\log n)$	$\Theta(1)$	$\Theta(n)$	$\Theta(\log n)$	$\Theta(1)$	$\Theta(n)$	$\Theta(\log n)$	$\log n$	n	$\log n$
BT	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\log n$	n	$\log n$
Min heap	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\log n$	$\log n$	$\log n$
AVL	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\log n$	$\log n$	$\log n$

Pre order ⟷ In order ⟷ Post order
$\Theta(n \log n)$

→ No. of labelled Trees of n vertices = n^{n-2}
" " rooted " " " " " = n^{n-1}
" " unlabelled " " " " = No formula
" " " B.T. " " " = $\frac{{}^{2n}C_n}{(n+1)}$
" " B.S.T. " " " = $\frac{{}^{2n}C_n}{(n+1)}$
" " labelled B.T. " " " = $\frac{{}^{2n}C_n}{(n+1)} \times n!$

Leaf node = Internal node $(n-1) + 1$ [for n any complete binary Tree]

→ AVL Tree (height Balanced BST):- $|H_L - H_R| \leq 1$

4 types of rotation:- LL (do clockwise rotation)
RR (" anticlockwise ")
LR (1st anticlockwise then clockwise)
RL (1st clockwise then anticlockwise)

Insertion → $\Theta(\log n)$ Deletion → $\Theta(\log n)$
Creation → $\Theta(n \log n)$
Rotation → $\Theta(n \log n)$